```
!pip install numpy pandas scikit-learn matplotlib

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
```

```
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (2.0.2)
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (2.2.2)
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-packages (1.6.1)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (3.10.0)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas) (2026.3)
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (1.16.3)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (1.5.3)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (3.6.0)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (4.63.0)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.5.0)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (26.2)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (11.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (3.3.2)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2->pandas) (1.
```

Geospatial concepts deal with data related to location on Earth. It is the study of “where things are and how they relate in space.” Uses coordinates like:

Latitude (North–South position) Longitude (East–West position)

Example:

A mountain area → specific location on map A battlefield → divided into zones Types of Geospatial Data

1. Vector Data Points (enemy location) Lines (routes) Polygons (battle zones)
2. Raster Data Images (satellite images, terrain maps)

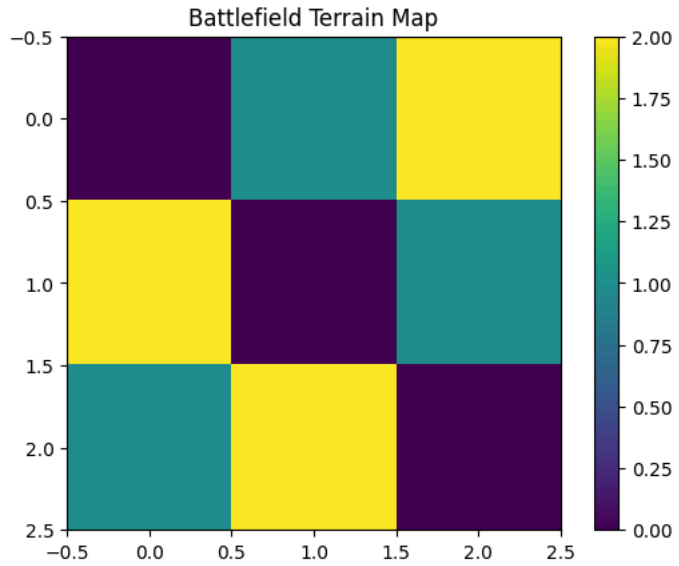
```
# Define terrain grid (0=Mountain, 1=Desert, 2=Urban)
terrain_map = np.array([
    [0, 1, 2],
    [2, 0, 1],
    [1, 2, 0]
])

terrain_labels = {0: "Mountain", 1: "Desert", 2: "Urban"}

print("Terrain Map:\n", terrain_map)
```

```
Terrain Map:
 [[0 1 2]
 [2 0 1]
 [1 2 0]]
```

```
plt.imshow(terrain_map)
plt.colorbar()
plt.title("Battlefield Terrain Map")
plt.show()
```




```
# Features: terrain, visibility, enemy_strength, troop_strength, distance
# Encoding: Low=0, Medium=1, High=2

data = [
    [0,0,2,0,1,"Hide"],
    [1,2,0,2,2,"Attack"],
    [2,1,1,1,1,"Defend"],
    [0,1,1,2,0,"Ambush"],
    [1,2,2,1,2,"Defend"],
    [2,0,0,2,1,"Attack"],
    [0,0,1,0,1,"Hide"],
    [1,1,2,1,2,"Defend"],
    [2,2,0,2,2,"Attack"],
    [0,1,2,0,0,"Ambush"]
]

df = pd.DataFrame(data, columns=[
    "Terrain","Visibility","Enemy","Troops","Distance","Strategy"
])

df
```

	Terrain	Visibility	Enemy	Troops	Distance	Strategy
0	0	0	2	0	1	Hide
1	1	2	0	2	2	Attack
2	2	1	1	1	1	Defend
3	0	1	1	2	0	Ambush
4	1	2	2	1	2	Defend
5	2	0	0	2	1	Attack
6	0	0	1	0	1	Hide
7	1	1	2	1	2	Defend
8	2	2	0	2	2	Attack
9	0	1	2	0	0	Ambush

```
X = df.drop("Strategy", axis=1)
y = df["Strategy"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

model = DecisionTreeClassifier()
model.fit(X_train, y_train)

accuracy = model.score(X_test, y_test)
print("Model Accuracy:", accuracy)
```

```
Model Accuracy: 1.0
```

```
def rule_based_logic(enemy, troops):
    if enemy > troops:
```

```
        return "Defend"
    return None
```

```
def predict_strategy(terrain, visibility, enemy, troops, distance):

    # Rule-based override
    rule = rule_based_logic(enemy, troops)
    if rule:
        return "Rule-Based Decision: " + rule

    # ML prediction
    prediction = model.predict([[terrain, visibility, enemy, troops, distance]])
    return "ML Prediction: " + prediction[0]
```

```
# Example input
terrain = 0       # Mountain
visibility = 0    # Low
enemy = 2         # High
troops = 1        # Medium
distance = 1

result = predict_strategy(terrain, visibility, enemy, troops, distance)
print("Suggested Strategy:", result)
```

```
Suggested Strategy: Rule-Based Decision: Defend
```

```
terrain = int(input("Terrain (0=Mountain,1=Desert,2=Urban): "))
visibility = int(input("Visibility (0=Low,1=Medium,2=High): "))
enemy = int(input("Enemy Strength (0=Low,1=Medium,2=High): "))
troops = int(input("Troop Strength (0=Low,1=Medium,2=High): "))
distance = int(input("Distance (0=Near,1=Medium,2=Far): "))

print(predict_strategy(terrain, visibility, enemy, troops, distance))
```

```
Terrain (0=Mountain,1=Desert,2=Urban): 2
Visibility (0=Low,1=Medium,2=High): 1
Enemy Strength (0=Low,1=Medium,2=High): 0
Troop Strength (0=Low,1=Medium,2=High): 0
Distance (0=Near,1=Medium,2=Far): 2
ML Prediction: Defend
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names,
  warnings.warn(
```